EXPERIMENT: PYTHON CODE GENERATION USING STRUCTURED PROMPTING

Aim

To design a structured prompt that instructs a Large Language Model (LLM) to generate a Python program for a specified computational problem, and to validate the generated code by executing it and checking the output.

Procedure

1. Open Google Colab at **colab.research.google.com** and create a new notebook.

2. Choose a simple computational problem: calculate the average of a list of numbers.

3. Write a structured prompt that clearly specifies the task, inputs, expected output, constraints, and validation requirements.

4. Ask an LLM to generate Python code based on the prompt. Review the code for correctness, readability, and edge cases.

5. Copy the generated code into a Colab code cell and run it.

6. Compare the actual output with the expected result. For the sample input [10, 20, 30, 40, 50], the expected average is 30.0.

7. Validate an edge case by passing an empty list and confirming that the function raises a ValueError with a clear message.

Structured Prompt

```
Role: You are a careful Python programmer.
Task: Write a Python function named calculate_average(numbers) that returns the arithmetic mean of a l
Inputs: A list of numeric values, for example [10, 20, 30, 40, 50].
Output: Return the average as a number.
Constraints:
1. Use Python 3.
2. Use sum() and len().
3. If the list is empty, raise ValueError with the message "The list cannot be empty".
4. Include a small test case and print the input list and calculated average.
Validation: Check the result for [10, 20, 30, 40, 50] and verify that the average is 30.0. Also test t
```

Program

```
def calculate_average(numbers):
    if not numbers:
        raise ValueError("The list cannot be empty")
    return sum(numbers) / len(numbers)


# Test with valid input
values = [10, 20, 30, 40, 50]
average = calculate_average(values)

print("Numbers:", values)
print("Average:", average)

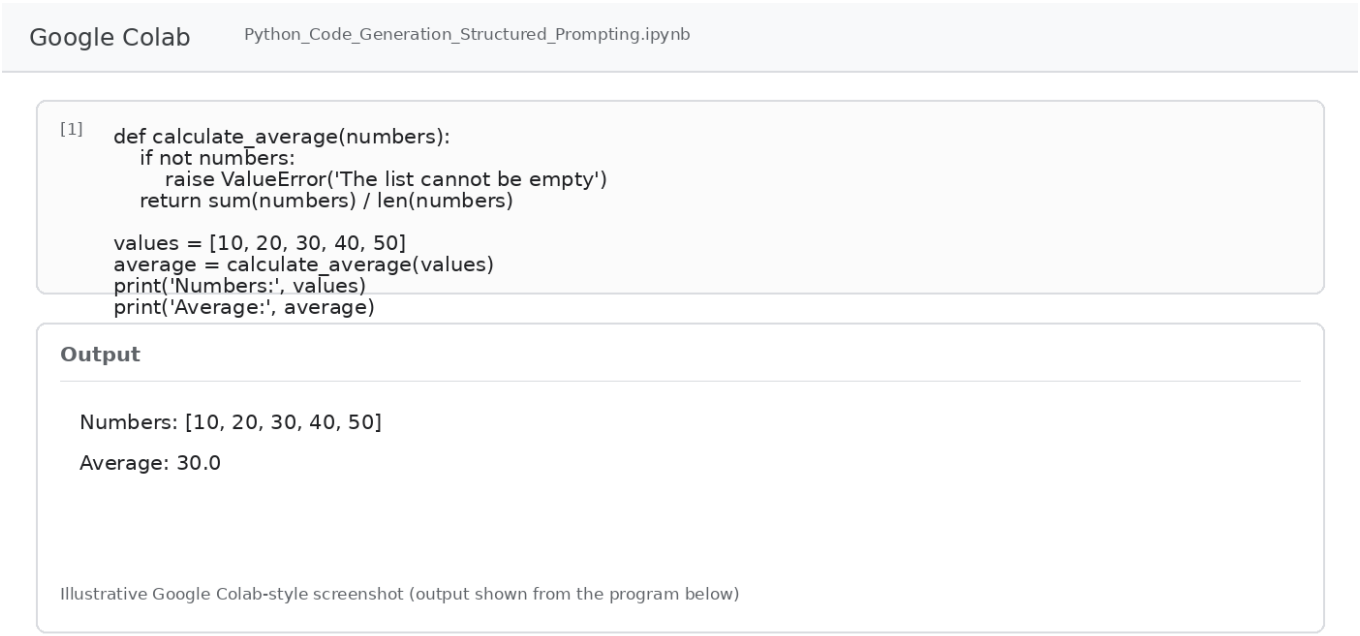
# Validate the result
assert average == 30.0
print("Validation passed: average is correct.")

# Test an edge case
```

```
try:
    calculate_average([])
except ValueError as error:
    print("Empty-list test passed:", error)
```

Google Colab Output Screenshot

The image below is an illustrative Colab-style screenshot. Run the program in your own Google Colab notebook to capture an authentic screenshot for submission.



Expected Output

```
Numbers: [10, 20, 30, 40, 50]
Average: 30.0
Validation passed: average is correct.
Empty-list test passed: The list cannot be empty
```

Result

A structured prompt was designed to specify the programming task, input, output, constraints, and validation criteria. The Python program calculated the average correctly and passed the normal-input and empty-list tests. Thus, structured prompting helps produce code that is clearer and easier to validate.

Note: The screenshot included is illustrative rather than a direct capture from a live Google Colab session. For an actual lab record, run the supplied program in Colab and replace it with your own screenshot if required.